Andrés Ruiz Palomino

Unity Developer · Gameplay · UI · Systems · C#

andres.gp.ruiz@gmail.com · (+51) 979-355-937 · arzpal.github.io · github.com/Arzpal · linkedin.com/in/arzpal · Peru (UTC-5)
English: Advanced · Open to full-time remote (USD) · Full US-hours overlap

SUMMARY

Unity and C# programmer with 3+ years in small indie teams, focused on gameplay systems and the feel layer on top of them: match-flow state machines, player controllers, camera rigs, hazards, pooling, and the UI architecture every screen routes through. Builds data-driven architecture where adding content means authoring data instead of writing code. Shipped on mobile, on PC and on physical arcade cabinets. Designs the systems he builds, a habit that comes from a game design degree and a game-design channel, so specs arrive buildable and edge cases get raised before they are written.

CORE SKILLS

Core	C#, .NET, Unity 6, URP, Addressables, Cinemachine, Input System, DOTween, FMOD, Git, NUnit / Unity Test Framework · GDScript / Godot
Gameplay & systems	Data-driven ScriptableObject architecture · composable effects · state machines and match flow · event-driven design · object pooling · UI architecture and screen routing · editor tooling (EditorWindow, GraphView / UIElements, custom property drawers)
Feel & design	Camera rigs, hitstop, screen shake, knockback & stagger, coyote time / jump buffering · systems design, technical design docs, playtest-driven iteration

EXPERIENCE

Game Programmer · DOB Studios — Mar 2026 – Aug 2026

"Runies Daycare: Merge & Pets" · live-service Unity 6 mobile game · 12-person team, ~118k LOC

- Root-caused the bug cluster in the project's first vertical slice: the tutorial had to switch gameplay systems on and off at exact moments, and the tutorial runner was what made that fragile. Proposed and built the replacement: small blocks holding one mini-action each, emitting their own events, each declaring its own condition for advancing.
- Built that model into an in-Unity node-graph editor (EditorWindow + GraphView / UIElements) with polymorphic actions via [SerializeReference]. Three of us author with it, and every standalone mechanic tutorial ships from it; forked a variation for story chapters (dialogue events + missions), so adding a feature or chapter means adding steps rather than writing a new flow.
- Built and owns the project's UI architecture: the manager every screen routes through (~1,145 lines), a filterable creature compendium with a custom multi-select dropdown, the exploration map (15 scripts covering locations, explorer states and popups), and resolution and scale management across device sizes.

Unity Developer · GDP Studios — Jul 2025 – Mar 2026

Unity games for physical casino cabinets, published under GMW · 4 titles shipped

- Shipped 4 titles onto a codebase that had to render across three cabinet models with completely different screen arrangements (three stacked 16:9; 16:9 / 9:16 / 16:9; 9:16 under 16:9), working inside those constraints on every title.
- Ran the reskin pipeline for cloned titles (same logic, different art) without letting logic drift between versions; drove optimization work, researched font optimization for their formats, and introduced an art/UX workflow that cut friction studio-wide.

Technical Artist → Lead Technical Artist → Studio Manager · Zarzilla Games — 2023 – 2025

- Redefined the technical art discipline at the studio: it had been art implementation only, and became a group that also owned the technical work adjacent to art, including tools that sped up the animation team's workflow. Defined technical pipelines and optimized runtime performance and resource usage.
- As Studio Manager: ran a studio of ~15 people across 5 areas (technical art, development, UI/raster art, animation, concept art); supervised development on Gin Rummy Super and two unannounced projects; coordinated schedules, sprint planning and cross-area handoffs. Stayed hands-on throughout: built the entire visual logic of Treasure Mines, a minigame inside Gin Rummy Super, still live in the shipped app.

SELECTED PROJECTS

full detail at arzpal.github.io

Tatemis3D · 3D creature-collector, real-time action combat · solo — Jul 2026 – Present

- Composable ability system where a creature is data, not code: abilities are lists of small reusable effects authored on ScriptableObjects. Built to absorb design churn, and it did, across repeated redesigns during development.
- Camera rig written from scratch instead of Cinemachine, to support three dynamic states with distinct behavior plus per-creature variations, and the feel layer on top: hitstop, screen shake, hit flash and damage popups scaled by the weight of the hit. Editor tooling for the combat lab, including a validator for the elemental type chart.

Don’t Chicken Out! · party game 2-4P · Raymi Games · lead programmer — Sep 2025 – Present

- Lead programmer on an 8-person team, author of roughly 70% of the game’s code, with design support alongside it. Match-flow state machine and rounds, local join for 2-4 players across split keyboard and gamepads, and an auto-rising camera rig.
- Designed and built rubber-band block pooling from a problem the lead designer raised: blocks are pooled per player rank (winning / neutral / losing) so what spawns tracks who is ahead. Platformer feel (jump buffering, coyote time, glide, fall multiplier) exposed through a ScriptableObject so the designer tunes handling directly. FMOD integrated from scratch.

Prismonaut · 2D platformer · founder of Naked Dog · 6-person core — Dec 2024 – May 2026

- Founded the studio and worked as designer, environment artist and lead technical artist, becoming the most active contributor on the repository and introducing several of the game-feel corrections to the character.
- Designed and programmed the game’s hazards (slimes, thorn roots, spikes, dirt-ball spawners, tutorial lava sequence), owning both sides of the tuning loop; present start to finish, from prototype through Game Jam Plus incubation to a Steam build.

Space Invaders-like · roguelike bullet-hell · Godot / GDScript · solo — 2024 · ~2 weeks

- Built alone in Godot around the time of the Unity licensing controversy, to evaluate another engine first-hand in case I had to move. The verdict was the point of the exercise: intuitive and fast to work in, but its scene-based architecture is simple enough that I would expect it to strain on a larger project.
- Designed the hitbox / hurtbox layer from the engine documentation rather than a tutorial, so bullets, asteroids and the player share one damage contract; debugged the projectile model (shots passing through targets, colliding with their own shooter, firing angle drifting from the ship) by normalizing projectile speed and separating collision layers. Sprites drawn by me.

EDUCATION

Toulouse Lautrec · Game Design & Digital Entertainment — 2023

ADDITIONAL

- Creator of a Spanish-language YouTube channel on game-design analysis; ongoing practice articulating design decisions, trade-offs and systems thinking.
- Playable builds on itch.io for Don’t Chicken Out!, Prismonaut and Crust and Conflict; gameplay video for six of the projects at arzpal.github.io.